

SpecFlow: Control-Anchored Spectral Flow Matching for Unpaired Single-Cell Perturbation Prediction

Anonymous Authors

Abstract

Predicting single-cell responses to genetic and chemical perturbations is essential for studying cellular regulation under interventions. However, perturbation datasets typically provide unpaired populations of control and perturbed cells rather than cell-level before-and-after trajectories, making transition-based modeling noisy under random control-target pairings. This difficulty is amplified for combinatorial and held-out perturbations, where a model must exploit gene-level structure rather than memorize condition labels. We propose SpecFlow, a control-anchored flow-matching framework that learns perturbation residual dynamics from measured control cells while conditioning the velocity field on Gene Ontology and control-derived coexpression spectra. SpecFlow combines three mechanisms: control-anchored flow matching to preserve source cell state, graph-spectral conditioning to encode functional and coexpression relationships among genes, and condition-wise optimal-transport coupling with delta-correlation training to reduce pairing noise and align with perturbation-effect metrics. We design an scDFM-aligned evaluation protocol on Norman additive and holdout splits, with control/additive baselines, component ablations, and planned mechanistic visualizations. **TODO: Insert verified SpecFlow results after running the provided scripts and collecting `agg\_results.csv` artifacts.**

1 Introduction

Single-cell perturbation assays make it possible to observe how cellular expression states change after genetic or chemical interventions. These assays are useful for studying regulatory programs and for prioritizing perturbations, especially when only a subset of possible single-gene and combinatorial interventions can be measured experimentally. The deployment constraint is practical: a perturbation predictor should generalize from observed perturbation populations to unseen or combinatorial conditions while preserving cell-state information from real control cells.

Existing perturbation predictors can be viewed as learning a conditional map from a control population and a perturbation descriptor to a perturbed expression distribution. This formulation is effective only when the model can infer a meaningful transition despite the fact that perturbation datasets are generally *unpaired*: control cells and perturbed cells are sampled from separate populations, not from matched before-and-after observations. Randomly pairing a control cell with a perturbed cell creates a noisy velocity target, while representing perturbations as condition identifiers can underuse functional relationships among targeted and downstream genes. Thus, the failure dimension we focus on is not generic predictive accuracy; it is the lack of a structurally informed and control-preserving transition model for unpaired perturbation populations.

Addressing this limitation is challenging for two reasons. First, unpaired cell populations do not provide ground-truth trajectories, so a transition model trained with arbitrary pairings may learn batch-level artifacts rather than perturbation effects. Second, perturbation response is gene-structured: combinatorial and held-out perturbations require information about where target genes sit in functional and coexpression networks. A method that addresses only the first challenge may

Placeholder for Figure 1. Left: random control-perturbed pairings produce noisy transition arrows in expression space. Right: SpecFlow anchors flows at measured control cells and orients the velocity field with GO/coexpression spectral structure. **TODO: Replace with schematic or result-grounded visualization.**

Figure 1: Motivating contrast for SpecFlow. The figure should show why unpaired random transitions are noisy and how control anchoring plus gene-structure conditioning addresses the failure mode.

preserve control states but fail to generalize structurally; a method that addresses only the second may encode gene structure but still train on implausible transitions.

Our key observation is that a perturbation response can be modeled as a *residual flow from a measured control state*, and the direction of this flow should be constrained by gene-structure information. Measured controls are not disposable noise samples: they provide the cell-state anchor from which a perturbation effect should be added. At the same time, perturbation masks identify target genes whose influence should be interpreted through functional and coexpression relationships. This suggests a simple design principle: learn a flow from control to perturbed expression, but condition the velocity field on graph-spectral gene coordinates and reduce unpaired-transition noise during training.

Motivated by this observation, we propose SpecFlow. SpecFlow first builds GO-derived and control-derived coexpression gene graphs, computes random-walk Laplacian spectral coordinates, and fuses them into per-gene structural features. It then learns a conditional velocity field that starts near measured control expression and predicts the residual path toward perturbed expression. During training, SpecFlow optionally aligns control and perturbed samples within the same perturbation condition using a Hungarian optimal-transport assignment and adds a delta-correlation objective that directly targets perturbation-effect direction. These components correspond to the two technical challenges: control anchoring and OT coupling address unpaired trajectories, while graph-spectral conditioning addresses structure-aware perturbation generalization.

The repository supports an scDFM-aligned evaluation plan on Norman additive and holdout protocols, with local control/additive baselines, external split loading, cell-eval-compatible outputs, and ablation configurations. However, the local checkout does not include data or completed SpecFlow runs, so this draft cannot claim empirical superiority. The experiments section is therefore written as a matched evidence plan, with TODO markers for all result claims.

Our contributions are:

1. **Problem framing.** We identify unpaired transition noise and missing gene-structure conditioning as two coupled obstacles in single-cell perturbation response prediction.
2. **Method.** We propose SpecFlow, a control-anchored flow-matching model whose velocity field is conditioned on perturbation masks and GO/coexpression spectral gene features.
3. **Training and evaluation protocol.** We implement condition-wise OT coupling, perturbation-delta correlation training, and an scDFM-compatible evaluation pipeline with planned additive, holdout, baseline, and ablation experiments. **TODO: Convert this protocol contribution into an empirical contribution after verified runs are available.**

2 Related Work

Single-cell perturbation prediction. Prior work on perturbation prediction aims to model cellular response under genetic or chemical interventions. The bundled baseline table references Control, Additive, scGPT, Geneformer, GEARS, CPA, STATE, CellFlow, and scDFM. These methods represent relevant comparison families: simple non-parametric baselines, perturbation-specific models, foundation-model baselines, and flow/diffusion-style models. **TODO: Add precise citations and method-specific descriptions after confirming the bibliography.**

Our work differs by focusing on a control-anchored transition objective for unpaired perturbation populations and by injecting GO/coexpression spectral structure into the velocity model. Unlike a pure condition-label approach, SpecFlow represents perturbations as gene-aligned masks interpreted through graph spectra. Unlike a generic distribution matcher, SpecFlow starts from measured control expression and learns residual dynamics.

Flow matching for conditional generation. Flow matching learns a velocity field that transports samples from a source distribution to a target distribution. SpecFlow uses this idea conditionally: the source is not pure noise but a noisy version of measured control expression. This design is intended to preserve source cell-state information while learning perturbation-induced residual change. **TODO: Add citations for conditional flow matching, rectified flow, and biological flow/diffusion models.**

Gene-structure representations. Gene perturbation effects are mediated by biological structure. SpecFlow uses two graph sources implemented in the repository: a Gene Ontology graph constructed from shared GO terms and a coexpression graph estimated from control cells. It then computes random-walk Laplacian spectral embeddings and applies sign-invariant encoders before graph fusion. **TODO: Add citations for GO-based gene representations, coexpression graph modeling, graph neural perturbation models, and spectral positional encodings.**

Benchmarking and evaluation protocols. Evaluation protocol matters because small differences in split, gene set, or metric computation can make perturbation results incomparable. SpecFlow writes `pred.h5ad` and `real.h5ad` files and invokes `cell\eval`, matching the scDFM-style evaluation interface. **TODO: Cite scDFM and the exact cell-eval protocol.**

3 Problem Setting

Let G be the number of modeled genes. A control cell is represented by $x_c \in \mathbb{R}^G$. A perturbation condition p is represented by a binary target mask $s_p \in \{0, 1\}^G$, where $s_{p,g} = 1$ indicates that gene g is directly targeted. For each condition p , the dataset provides a population of perturbed cells $\mathcal{X}_p = \{x_p^{(i)}\}$ and a shared control population $\mathcal{X}_c = \{x_c^{(j)}\}$.

The goal is to learn a conditional generator

$$\hat{x}_p \sim q_\theta(\cdot \mid x_c, s_p)$$

that maps sampled control cells and perturbation masks to predicted perturbed expression. The setting is unpaired: there is no observed trajectory from a particular $x_c^{(j)}$ to a particular $x_p^{(i)}$.

The primary biological quantity is the perturbation effect:

$$\Delta_p = \mathbb{E}[x_p \mid p] - \mathbb{E}[x_c], \quad \hat{\Delta}_p = \mathbb{E}[\hat{x}_p \mid p] - \mathbb{E}[x_c].$$

Placeholder for Figure 2. Pipeline: graph construction $\rightarrow$ spectral gene features $\rightarrow$ control-anchored flow matching with optional OT coupling $\rightarrow$ Euler sampling and cell-eval outputs. **TODO:** Replace with a clean method diagram.

Figure 2: Overview of SpecFlow. Each block corresponds to one challenge: unpaired trajectories, structure-aware perturbation conditioning, and metric-aligned training.

Table 1: Challenge-to-component mapping in SpecFlow.

Challenge	Component		Code evidence
No paired cell trajectories	Control-anchored flow match-	ing	<code>flow_matching.py</code>
Random pairing creates noisy velocity targets	Condition-wise OT coupling		<code>_ot_align_targets</code>
Perturbations require gene-structure context	GO/coexpression conditioning	spectral	<code>graph/*.py,</code> <code>spectral_fusion.py</code>
Metrics focus on perturbation effects	Delta-correlation loss	auxiliary	<code>SpecFlowTrainer._delta_correlation_</code>

The repository emphasizes `pearson\_delta`, the Pearson correlation between Δ_p and $\hat{\Delta}_p$, as the main effect-level metric. Additional configured or planned metrics include MSE, MAE, DE Spearman, distributional similarity (DS), L2, Pearson Delta-hat, and Pearson Delta-hat20.

This differs from standard paired transition modeling because the true source-target cell correspondence is unavailable. It also differs from condition-only prediction because the perturbation descriptor is gene-aligned and can be interpreted through biological graph structure.

4 Method

4.1 Overview

Given control expression x_c , perturbation mask s_p , and gene graphs, SpecFlow:

1. builds GO and coexpression graph spectra;
2. encodes per-gene structural features and perturbation features;
3. predicts a velocity field over expression space;
4. trains the velocity with control-anchored flow matching, optionally after condition-wise OT coupling;
5. samples perturbed cells by Euler integration from a noisy control anchor.

4.2 Graph-Spectral Gene Conditioning

The goal of graph-spectral conditioning is to give the velocity field a gene-level structural coordinate system. Directly treating a perturbation as an arbitrary condition token is insufficient for held-out or combinatorial perturbations because it does not expose functional proximity or coexpression relationships among genes.

SpecFlow constructs two graphs. The GO graph connects genes using shared Gene Ontology terms and a top- k Jaccard graph. The coexpression graph is built from absolute Pearson correlations among genes in control cells. For each graph, SpecFlow computes random-walk Laplacian spectral coordinates by solving the symmetric normalized Laplacian and converting to a random-walk basis.

Let U^{GO} and U^{coexp} be the two spectral embeddings. Because eigenvector signs are arbitrary, SpecFlow applies SignNet encoders before fusion. `DualGraphSpectralFusion` then performs macro/micro scale fusion and cross-graph fusion, producing per-gene structural features:

$$Z_p = \text{Fuse}(U^{GO}, U^{coexp}, s_p).$$

This design encourages perturbation predictions to depend on where targeted genes lie in functional and expression-derived graph geometry. **TODO: Validate this claim with graph-none, GO-only, coexpression-only, and no-spectral-embedding ablations.**

4.3 Perturbation-Conditioned Velocity Field

The goal of the velocity field is to predict how every gene’s expression should move at interpolation state x_t . For each gene g , the model builds a token from local control expression, spectral features, mask value, and a perturbation embedding:

$$h_g = \text{GeneTokenEncoder}(x_{c,g}, Z_{p,g}, s_{p,g}, e_p).$$

An attentive pooling module aggregates gene tokens into a cell-level condition vector. The velocity field combines local per-gene features $(x_{t,g}, x_{c,g}, Z_{p,g}, s_{p,g})$ with global conditioning from the pooled cell representation, time embedding, and perturbation embedding. Residual MLP blocks and FiLM-style modulation produce the final velocity:

$$v_\theta(x_t, t, x_c, s_p, Z_p) \in \mathbb{R}^G.$$

The default Norman configs enable spectral propagation and use the default `legacy` perturbation encoder plus `spectral` propagation variant. The repository also implements a `graph\_pool` perturbation encoder and `contextual\_local` propagation route, but these are not enabled in the default Norman configs. **Need confirmation: Choose whether the paper’s final main method is the 0602 default protocol or the newer contextual-local variant.**

4.4 Control-Anchored Flow Matching

The goal of control anchoring is to preserve measured source cell state while learning perturbation residuals. SpecFlow samples a time $t \sim \mathcal{U}(0, 1)$ and defines:

$$x_0 = x_c + \sigma\epsilon, \quad x_t = (1 - t)x_0 + tx_1, \quad u_t = x_1 - x_0,$$

where x_1 is a perturbed target sample and ϵ is Gaussian noise. The primary loss is:

$$\mathcal{L}_{FM} = \mathbb{E} \left[\|v_\theta(x_t, t, x_c, s_p, Z_p) - u_t\|_2^2 \right].$$

The first term in the path definition keeps the source near a real control cell, while the target velocity learns the residual effect needed to reach perturbed expression. Thus, minimizing this objective encourages a control-preserving perturbation flow rather than unconditional expression generation.

Table 2: Logic-level training and inference procedure. This table summarizes method logic rather than implementation details such as caching, checkpointing, AMP, and EMA.

Step	Procedure
1	Build GO and coexpression gene graphs.
2	Compute graph spectral embeddings and fused gene features.
3	Sample a perturbation condition, control cells, and perturbed cells.
4	If OT coupling is enabled, reorder perturbed cells by Hungarian assignment within condition.
5	Sample t and construct x_0 , x_t , and target velocity $x_1 - x_0$.
6	Predict velocity conditioned on x_c , s_p , and graph spectra.
7	Update parameters with flow-matching loss and optional delta-correlation loss.
8	At inference, initialize from $x_c + \sigma\epsilon$, integrate with Euler updates, and write prediction artifacts for cell-eval.

4.5 Condition-Wise OT Coupling

The goal of OT coupling is to reduce noisy training velocities caused by arbitrary control-target pairings. Within each perturbation condition in a minibatch, SpecFlow computes pairwise squared distances between control and perturbed expression vectors and solves a Hungarian assignment. The perturbed samples are reordered according to this assignment before the flow target is computed.

This coupling does not recover true cell-level trajectories, which are unobserved. It is a mini-batch approximation intended to make training pairs more expression-compatible. **TODO: Validate its contribution with `flow.ot\_coupling=false` ablations.**

4.6 Delta-Correlation Training

The main biological metric compares perturbation-effect directions rather than only per-cell reconstruction. SpecFlow therefore optionally adds a condition-grouped correlation loss:

$$\mathcal{L}_\Delta = 1 - \text{corr} \left(\mathbb{E}_{i \in p} [v_\theta^{(i)}], \mathbb{E}_{i \in p} [x_1^{(i)} - x_0^{(i)}] \right).$$

The complete implemented training objective is:

$$\mathcal{L} = \mathcal{L}_{FM} + \lambda_\Delta \mathcal{L}_\Delta + \lambda_{MMD} \mathcal{L}_{MMD},$$

where the default Norman config sets $\lambda_\Delta = 0.03$ and $\lambda_{MMD} = 0$. The delta term aligns training with perturbation-effect correlation; the MMD term exists in code but is disabled in the default Norman configs. **TODO: Validate with no-delta-correlation and MMD-on ablations.**

4.7 Training and Inference Algorithm

5 Experiments

Following the AAAI oral-paper guide, experiments should be organized around research questions rather than a single leaderboard table. The current repository supports the following plan, but local SpecFlow results are not present.

5.1 Research Questions

Q1: Effectiveness. Does SpecFlow improve perturbation-effect prediction on scDFM-aligned Norman additive splits compared with simple and strong baselines? **TODO:** Run SpecFlow fold 0–4 and compare against clearly labeled Control, Additive, and paper-reported or rerun model baselines.

Q2: Generalization. Does SpecFlow generalize to held-out single and double perturbations? **TODO:** Run `configs/norman\_holdout.yaml` and summarize Single/Double subsets.

Q3: Necessity. Which components are necessary: graph spectra, spectral propagation, OT coupling, delta-correlation loss, and control anchoring? **TODO:** Run component ablations.

Q4: Practicality and interpretability. What are the computational costs, and do predictions/propagation patterns support the intended mechanism? **TODO:** Report runtime/memory and generate UMAP, delta scatter, top-DE boxplot, and graph propagation figures.

5.2 Experimental Setup

Benchmarks. The primary benchmark is Norman CRISPRa perturbation prediction under additive and holdout protocols. The repository also includes ComboSciPlex configuration and drug-to-target mapping files. **TODO:** Confirm exact dataset statistics after data are available.

Splits. `configs/norman.yaml` uses `data/split\_results.pkl` for scDFM-aligned additive folds. `configs/norman\_holdout.yaml` uses `data/splits/norman\_holdout.pkl`, generated by `scripts/build\_holdout.py`. **TODO:** Verify split files in the run environment.

Baselines. Local scripts support Control and Additive baselines. The bundled `paper\_baselines/scdfm\_report.py` includes paper-reported metrics for multiple baselines, but these are not local reruns. **TODO:** Decide whether main tables use rerun baselines or clearly labeled paper-reported values.

Metrics. Main metrics should include Pearson Delta, MSE, MAE, DE Spearman, and DS. Additional scDFM-aligned metrics include L2, Pearson Delta-hat, and Pearson Delta-hat20.

Implementation. The default Norman config uses `batch\_size=512`, `max\_steps=200000`, `learning\_rate=0.001`, `sigma=0.2`, `ot\_coupling=true`, `delta\_corr\_weight=0.03`, and `inference.ode\_steps=30`. **TODO:** Add hardware, software, random seed, and wall-clock details from logs.

5.3 Main Results: Norman Additive

Table 3 should answer Q1. It must report mean and standard deviation across folds, not a single best run. Because no local SpecFlow outputs exist, all SpecFlow cells remain TODO.

The result paragraph should only be written after data are available. **TODO:** Replace with: Table 3 shows that [verified result].

Table 3: Norman additive main results under scDFM-aligned evaluation. Higher is better for Pearson Delta, DE Spearman, and DS; lower is better for MSE and MAE. All SpecFlow entries must be filled from local `agg\_results.csv` files.

Method	Source	Pearson Delta	MSE	MAE	DE Spearman	DS
Control	TODO/rerun	TODO	TODO	TODO	TODO	TODO
Additive	TODO/rerun	TODO	TODO	TODO	TODO	TODO
scDFM	paper/rerun?	TODO	TODO	TODO	TODO	TODO
CellFlow	paper/rerun?	TODO	TODO	TODO	TODO	TODO
SpecFlow	local rerun	TODO	TODO	TODO	TODO	TODO

Table 4: Norman holdout generalization.

Subset	Method	Pearson Delta	MSE	MAE	DE Spearman	DS
Single	SpecFlow	TODO	TODO	TODO	TODO	TODO
Double	SpecFlow	TODO	TODO	TODO	TODO	TODO

5.4 Generalization: Norman Holdout

Table 4 should answer Q2 by reporting held-out single and double perturbation results. This is more important for a strong paper than additive-only interpolation.

TODO: Compare to scDFM paper-reported or rerun holdout baselines with source labels.

5.5 Ablation Study

Table 5 should answer Q3. Each row maps to a specific claim.

Ablation conclusions must be component-specific. For example, do not write “all components are useful” unless every row supports it.

5.6 Robustness, Efficiency, and Visualization

Robustness. ComboSciPlex can test whether the mechanism extends beyond Norman CRISPRa.

TODO: Run `configs/experiments/combosciplex\_4gpu.yaml` only after Norman results are stable.

Efficiency. Report training time, inference time, GPU memory, and overhead from OT coupling.

TODO: Collect from logs.

Visualization. Figure 1 should show the motivating failure mode. Additional figures should show predicted versus true perturbation deltas, UMAP or top-DE gene behavior, and graph-propagation heatmaps if spectral/contextual propagation is claimed as mechanistic evidence.

6 Discussion and Limitations

This draft’s main limitation is evidential: the local repository does not include completed experiments. Thus, the current paper can describe SpecFlow and its evaluation plan, but it cannot claim performance improvements.

Table 5: Component ablations on Norman additive or holdout.

Variant	Pearson Delta	MSE	MAE	DE Spearman	DS
Full SpecFlow	TODO	TODO	TODO	TODO	TODO
w/o graph	TODO	TODO	TODO	TODO	TODO
w/o spectral propagation	TODO	TODO	TODO	TODO	TODO
w/o OT coupling	TODO	TODO	TODO	TODO	TODO
w/o delta correlation	TODO	TODO	TODO	TODO	TODO
w/o control anchor	TODO	TODO	TODO	TODO	TODO

The second limitation is method-version ambiguity. The README highlights `graph\_pool` perturbation encoding and `contextual\_local` propagation, while the default Norman configs use the legacy perturbation encoder and spectral propagation. Before submission, the paper, README, and configs must agree on the final main method.

Third, SpecFlow depends on external biological structure. GO annotations may be incomplete, and coexpression graphs estimated from control cells may be dataset-specific. This does not invalidate the core control-anchored flow formulation, but it affects the scope of graph-structure claims.

Fourth, minibatch OT coupling is an approximation to unobserved cell matching. It reduces arbitrary pairing by expression distance, but it does not prove biological lineage or trajectory correctness.

Fifth, the velocity field is mainly a local per-gene MLP with global conditioning and propagation features. It may not capture all higher-order gene interactions that a full cross-gene attention model could represent.

7 Conclusion

We studied unpaired single-cell perturbation response prediction, where models must infer perturbation effects without observed cell-level trajectories. SpecFlow addresses this setting by anchoring flow matching at measured control cells, conditioning the velocity field on GO/coexpression spectral gene structure, and reducing random-pairing noise through condition-wise OT coupling and delta-correlation training. The repository provides a concrete implementation and an scDFM-aligned evaluation plan. The next step is empirical: run the planned additive, holdout, baseline, ablation, robustness, and visualization experiments, then replace all TODO placeholders with verified evidence.

AAAI-Oral Guide Compliance Checklist

Sharp problem. Current framing: unpaired perturbation populations create noisy transition targets, and perturbation response requires gene-structure conditioning.

Mechanistic insight. Current insight: perturbation response should be modeled as a control-anchored residual flow whose direction is conditioned by gene graph spectra.

Minimal method. Current modules are graph-spectral conditioning, control-anchored flow matching, OT coupling, and delta-correlation training. The final paper should keep each module tied to

a challenge to avoid looking like a module stack.

Matched evidence. Main evidence is still missing: additive results, holdout results, ablations, robustness, efficiency, and visualizations. The manuscript is therefore not submission-ready.

Claims allowed now. SpecFlow implements control-anchored flow matching, GO/coexpression spectral conditioning, condition-wise OT coupling, delta-correlation training, and scDFM-style output generation.

Claims not allowed yet. Do not claim that SpecFlow outperforms baselines, generalizes better to held-out perturbations, proves graph spectra/OT/delta effectiveness, or is efficient/robust/practical without runtime and result evidence.